



TASK 5 REPORT

PHISHAWARE

PHISHING AWARENESS AND INCIDENT RESPONSE SIMULATION

NAME: AHAMED S

INTERNSHIP ORGANIZATION: APEXPLANET

TASK: TASK 5

DURATION: DAY 49 – DAY 360

DATE: OCTOBER 2026

1. INTRODUCTION

Email is widely used for communication, but it can also be used to distribute phishing messages. These messages may create urgency, request account verification, or encourage users to follow unexpected links.

PhishAware demonstrates a basic approach to identifying these warning signs through rule-based text analysis. It also presents a structured response to a simulated phishing incident, including detection, analysis, containment, eradication, recovery, and lessons learned.

The project uses fictional data and does not target real users or collect credentials.

2. PROBLEM STATEMENT

Users may overlook suspicious characteristics in emails, particularly when messages use urgent wording or imitate account-security notifications. An awareness tool can help highlight indicators that require further investigation.

PhishAware addresses this problem by analyzing a sample email and documenting a simulated incident-response process.

3. OBJECTIVES

- Identify common phishing indicators in email text.
- Develop a basic Python-based email analysis program.
- Demonstrate incident-response stages.
- Record simulated incident findings in a log file.
- Explain preventive measures and safe email practices.

4. TOOLS AND TECHNOLOGIES USED

Tool	Purpose
Kali Linux	Operating system used for development and testing
Python 3	Implements the detector and incident-response simulator
Linux Terminal	Creates files and executes programs
Text Editor	Creates and edits project files
Mermaid	Defines the incident-response workflow diagram
Microsoft Word	Formats the final report

5. PROJECT STRUCTURE

The project contains the following files and folders:

- `phishing_detector.py` — analyzes the fictional email for suspicious indicators.
- `incident_response.py` — simulates the incident-response stages.
- `samples/` — stores the mock phishing email.
- `evidence/` — stores the incident-response log.
- `screenshots/` — stores demonstration evidence.
- `diagrams/` — contains the workflow diagram source.
- `README.md` — provides the project overview and instructions.
- `report.md` — contains the report draft.
- `presentation-script.md` — contains the presentation script.

6. SYSTEM WORKFLOW

The project begins with a fictional email sample. The Python detector checks the text for selected indicators and displays the findings. These findings are reviewed manually, and the incident-response simulator demonstrates how a suspicious message would be handled in a controlled exercise.

The workflow ends with documentation of the incident and recommended preventive measures.

7. IMPLEMENTATION

7.1 Creation of the Sample Email

A fictional phishing-style email was created for testing. The message includes urgent wording, an account-verification request, and a reserved example URL. The sample provides controlled input for the detector.

Run:

```
cat ~/PhishAware/samples/mock_phishing_email.txt
```

7.2 Phishing Indicator Detection

The `phishing_detector.py` program reads the sample email and converts the text to lowercase. It checks the text against predefined keywords grouped into three categories:

1. Urgent or threatening language.
2. Requests for account verification.
3. Presence of a URL.

If a keyword from a category is found, the program displays an alert for that category. The program counts the categories that triggered alerts and recommends manual review.

7.3 Execution of the Detector

The detector is executed from the project directory using:

```
python3 phishing_detector.py
```

The program displays the analysis results in the terminal.

8. INCIDENT RESPONSE SIMULATION

The `incident_response.py` program demonstrates a response to the fictional incident. It prints the response stages and saves a timestamped log to `evidence/incident-response-log.txt`.

8.1 Detection

The fictional email is flagged because it contains urgent language, an account-verification request, and a URL.

8.2 Analysis

The detected indicators are reviewed to understand why the message may be suspicious. These indicators alone do not establish that an email is malicious.

8.3 Containment

The simulated response treats the message as suspicious and avoids opening the included link.

8.4 Eradication

The exercise records the removal of the mock email from the simulated inbox. No real mailbox is accessed or modified.

8.5 Recovery

The simulated recovery step recommends checking account status through the official website or application rather than using a link from the message.

8.6 Lessons Learned

The exercise highlights the importance of verifying sender details, avoiding unexpected links, reporting suspicious emails, and improving user awareness.

Run `python3 incident_response.py` and capture the output showing all six stages.

Run:

```
cat ~/PhishAware/evidence/incident-response-log.txt
```

Capture the incident ID, timestamp, and recorded response stages.

9. WORKFLOW DIAGRAM

The workflow represents the main activities performed by the project:

1. A fictional phishing email is supplied as input.
2. The Python detector analyzes the email text.
3. Suspicious indicators are flagged for manual review.
4. The message is analyzed without opening its link.
5. The simulated process records containment, eradication, and recovery.
6. The final outcome and lessons learned are documented.

The workflow separates automated indicator checking from manual review and simulated response activities.

Mock Phishing Email → Python Email Detector → Suspicious Indicators Found? → Manual Review → Safe Containment → Simulated Eradication → Recovery → Incident Log and Lessons Learned

10. RESULTS AND ANALYSIS

The fictional email sample produced three indicator alerts during testing.

Indicator Category	Result
Urgent or threatening language	Alert
Account-verification request	Alert
URL present in message	Alert

The detector displayed a recommendation for manual review. The incident-response simulator then displayed the six response stages and generated a log file.

The results show that the prototype can identify the selected keywords in the supplied sample and document the simulated response. The test does not measure detection accuracy across a large dataset and does not demonstrate detection of all phishing emails.

```
Session Actions Edit View Help
(kali㉿kali)-[~]
$ mkdir -p ~/PhishAware
cd ~/PhishAware
$ mkdir -p samples evidence screenshots diagrams
$ ls
diagrams  evidence  incident_response.py  phishing_detector.py  samples  screenshots
$ cd ~/PhishAware
cat > samples/mock_phishing_email.txt <<'EOF'
From: Security Team <security-alert@example.invalid>
Subject: URGENT - Verify Your Account Immediately

Dear User,

Your account requires immediate verification.
Failure to verify may result in temporary account suspension.

Visit the following training link:
https://example.invalid/verify

This is a fictional phishing-awareness sample created for educational purposes.
EOF
(kali㉿kali)-[~/PhishAware]
$ cat samples/mock_phishing_email.txt
From: Security Team <security-alert@example.invalid>
Subject: URGENT - Verify Your Account Immediately

Dear User,

Your account requires immediate verification.
Failure to verify may result in temporary account suspension.

Visit the following training link:
https://example.invalid/verify

This is a fictional phishing-awareness sample created for educational purposes.
(kali㉿kali)-[~/PhishAware]
$ >....
    raise SystemExit(1)

email = email_file.read_text().lower()

checks = {
```

```
Session Actions Edit View Help

    "Urgent or threatening language": [
        "urgent", "immediately", "suspension"
    ],
    "Requests account verification": [
        "verify your account", "account requires"
    ],
    "Contains a URL": [
        "http://", "https://"
    ]
}

print("=" * 45)
print("      PHISHAWARE - EMAIL ANALYSIS")
print("=" * 45)

alerts = 0

for name, keywords in checks.items():
    found = any(word in email for word in keywords)
    print(f"[{'ALERT' if found else 'OK'}]_{name}")
    if found:
        alerts += 1

print("-" * 45)
print(f"Indicators detected: {alerts}")
print("Result: Manual review recommended.")
print("Note: Indicators alone do not prove phishing.")
PY

(kali@kali)-[~/PhishAware]
$ python3 phishing_detector.py

=====
PHISHAWARE - EMAIL ANALYSIS
=====
[ALERT] Urgent or threatening language
[ALERT] Requests account verification
[ALERT] Contains a URL

Indicators detected: 3
Result: Manual review recommended.
Note: Indicators alone do not prove phishing.

(kali@kali)-[~/PhishAware]
$ >.....
print("      PHISHAWARE - INCIDENT RESPONSE SIMULATOR")
print("=" * 55)

incident_id = "PH-001"

steps = [
    ("DETECTION", "Suspicious email flagged by the detector."),
```

```
Session Actions Edit View Help

("ANALYSIS", "Urgent wording, verification request, and URL reviewed."),
("CONTAINMENT", "Email marked suspicious; link not opened."),
("ERADICATION", "Mock email removed from the simulated inbox."),
("RECOVERY", "Account status checked through the official website."),
("LESSONS LEARNED", "Improve email verification and user awareness.")
]

with open("evidence/incident-response-log.txt", "w") as log:
    log.write("PHISHAWARE - INCIDENT RESPONSE REPORT\n")
    log.write(f"Incident ID: {incident_id}\n")
    log.write(f>Date: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n")
    log.write("Type: Simulated phishing-awareness incident\n\n")

    for stage, description in steps:
        print(f"\n[{stage}]")
        print(description)
        log.write(f"{stage}: {description}\n")

    log.write("\nOutcome: Simulation completed.\n")
    log.write("No real accounts or credentials were involved.\n")

print("\n" + "=" * 55)
print("Simulation completed.")
print("Log saved to evidence/incident-response-log.txt")
print("=" * 55)
PY

(kali@kali)-[~/PhishAware]
$ python3 incident_response.py

=====
PHISHAWARE - INCIDENT RESPONSE SIMULATOR
=====

[DETECTION]
Suspicious email flagged by the detector.

[ANALYSIS]
Urgent wording, verification request, and URL reviewed.

[CONTAINMENT]
Email marked suspicious; link not opened.

[ERADICATION]
Mock email removed from the simulated inbox.

[RECOVERY]
Account status checked through the official website.

[LESSONS LEARNED]
Improve email verification and user awareness.
```

```
kali@kali: ~$ cat evidence/incident-response-log.txt
Simulation completed.
Log saved to evidence/incident-response-log.txt

(kali@kali)-[~/PhishAware]
$ cat evidence/incident-response-log.txt
PHISHAWARE - INCIDENT RESPONSE REPORT
Incident ID: PH-001
Date: 2026-10-09 12:16:01
Type: Simulated phishing-awareness incident

DETECTION: Suspicious email flagged by the detector.
ANALYSIS: Urgent wording, verification request, and URL reviewed.
CONTAINMENT: Email marked suspicious; link not opened.
ERADICATION: Mock email removed from the simulated inbox.
RECOVERY: Account status checked through the official website.
LESSONS LEARNED: Improve email verification and user awareness.

Outcome: Simulation completed.
No real accounts or credentials were involved.

(kali@kali)-[~/PhishAware]
$ cd ~/PhishAware

cat > diagrams/phishing-incident-workflow.mmd <<'EOF'
flowchart TD
    A[Mock Phishing Email] --> B[Python Email Detector]
    B --> C{Suspicious Indicators?}
    C -->|Yes| D[Flag for Manual Review]
    C -->|No| E[No Indicators Detected]
    D --> F[Analyze Sender, Wording and URL]
    F --> G[Contain: Do Not Open Link]
    G --> H[Eradicate: Remove Mock Email]
    H --> I[Recover: Verify Through Official Channel]
    I --> J[Document Incident and Lessons Learned]
    E --> J
EOF

(kali@kali)-[~/PhishAware]
$ cat > diagrams/README.md <<'EOF'
# PhishAware Workflow Diagram

This diagram shows the stages of the phishing-awareness project.

1. A fictional email is analyzed by the Python detector.
2. Suspicious indicators are flagged for manual review.
3. The simulated incident is contained.
4. The mock email is removed.
5. Recovery and lessons learned are documented.
```

```
kali@kali: ~/PhishAware
Session Actions Edit View Help
This is an educational simulation. No real phishing campaign is deployed.
EOF

(kali@kali) ~/PhishAware
$ ls -l diagrams
total 8
-rw-rw-r-- 1 kali kali 429 Oct 9 12:16 phishing-incident-workflow.mmd
-rw-rw-r-- 1 kali kali 403 Oct 9 12:16 README.md

(kali@kali) ~/PhishAware
$ >....
## 6. Incident Response
The simulation demonstrates:
1. Detection of a suspicious email.
2. Analysis of suspicious characteristics.
3. Containment by not opening the link.
4. Removal of the mock email.
5. Recovery through an official channel.
6. Documentation of lessons learned.

## 7. Mitigation Measures
- Verify the sender's full email address.
- Avoid unexpected links and attachments.
- Access accounts through official websites or applications.
- Enable multi-factor authentication.
- Report suspicious emails to the appropriate security team.
- Provide regular phishing-awareness training.
- Use email filtering and security monitoring.

## 8. Results
The sample email produced three indicator alerts: urgent language, an account-verification request, and a URL. The incident-response simulator completed all six documented stages.

## 9. Limitations
This is a basic keyword-based prototype. It does not use machine learning, inspect email headers, verify URL reputation, or guarantee detection of every phishing message.

## 10. Ethical Considerations
This project uses fictional data and a simulated response. It does not target real users, collect credentials, or deploy phishing messages.

## 11. Conclusion
PhishAware demonstrates basic phishing awareness, rule-based email analysis, and incident-response documentation. It highlights the importance of manual verification and safe email-handling practices.
EOF

(kali@kali) ~/PhishAware
$ cd ~/PhishAware
head -n 12 README.md
# PhishAware - Phishing Awareness and Incident Response

## 1. Project Overview
```

11. MITIGATION AND PREVENTION

The following measures can help reduce phishing risk:

- **Verify the sender:** Review the complete email address and confirm unexpected requests through a trusted channel.
- **Avoid suspicious links:** Do not open unexpected links or attachments.
- **Use official channels:** Access accounts through trusted bookmarks or official applications.
- **Enable multi-factor authentication:** Add another layer of account protection.
- **Report suspicious messages:** Forward concerns to the appropriate IT or security team.
- **Provide awareness training:** Teach users to recognize social-engineering tactics.
- **Use email security controls:** Filtering and monitoring can help identify suspicious messages.

12. LIMITATIONS

The prototype has several limitations:

1. It uses predefined keywords rather than advanced machine-learning techniques.
2. It does not inspect full email headers or verify URL reputation.
3. It may flag legitimate messages that contain similar wording.
4. It may miss phishing messages that do not contain the selected keywords.
5. The incident-response process is simulated and does not perform real mailbox remediation.
6. Testing uses a single fictional sample rather than a large collection of labeled emails.

Therefore, the tool should be treated as an educational demonstration and not as a production-grade phishing detection system.

13. ETHICAL CONSIDERATIONS

The project uses fictional sample data in a controlled learning environment. No real users are targeted, no credentials are collected, and no live phishing messages are deployed. The response process is simulated for awareness and educational purposes.

14. FUTURE ENHANCEMENTS

Possible improvements include:

- Testing with a larger set of safely collected and labeled email samples.
- Adding email-header analysis.
- Integrating trusted URL reputation services where permitted.
- Evaluating false-positive and false-negative rates.
- Developing a graphical interface for displaying analysis results.
- Improving the incident log with structured fields and reporting.

These are potential future improvements and are not implemented in the current prototype.

15. CONCLUSION

PhishAware demonstrates basic phishing-indicator detection and a simulated incident-response workflow using Python on Kali Linux. The detector identifies selected warning signs in a fictional email, while the simulator documents the response stages and lessons learned.

Although the prototype is limited to simple keyword matching, it illustrates the value of manual review, safe email practices, incident documentation, and cybersecurity awareness.

16. REFERENCES

1. Python Software Foundation. *Python 3 Documentation*. <https://docs.python.org/3/>
2. National Institute of Standards and Technology (NIST). *Computer Security Incident Handling Guide, SP 800-61 Rev. 2*. <https://csrc.nist.gov/pubs/sp/800/61/r2/final>
3. Cybersecurity and Infrastructure Security Agency (CISA). *Recognize and Report Phishing*. <https://www.cisa.gov/secure-our-world/recognize-and-report-phishing>

APPENDIX A: COMMANDS USED

The following commands were used to execute and inspect the project:

```
python3 phishing_detector.py
```

```
python3 incident_response.py
```

```
cat evidence/incident-response-log.txt
```

```
cat samples/mock_phishing_email.txt
```

```
find . -maxdepth 3 -type f -print
```